

Chapter 4

Hierarchy and Polymorphism

With the concepts introduced so far, there has been no technical reason to prefer having an instance method that belongs to an object over a static method that takes that object as a parameter. There's a logical reason, that if a method queries or manipulates the internal state of an object, it should be an instance method of that object, but that's a matter of style not of technical importance. In this chapter, we discuss the concept of two classes existing in a *subclass* relationship, where every instance of the subclass is also considered to be an instance of the parent superclass. To demonstrate this, we will use a concept from geometry: every square is a rectangle (but not every rectangle is a square!) and a square has an extra invariant that a rectangle may not satisfy, namely that its width and its height are equal. The subclass does more work to maintain this invariant.

4.1 Defining a Subclass

A *subclass* relationship models the situation in which all instances of the subclass are necessarily instances of the parent superclass. This is sometimes called an “is-a” relationship, as in saying that a square is a rectangle. In Java, this is stated using the “**extends**” keyword, as in the following example.

<shapes/Rectangle.java 41a>≡ 41a

```
public class Rectangle {  
    <fields of the shapes/Rectangle class 42a>  
    <constructors of the shapes/Rectangle class 42b>  
    <methods of the shapes/Rectangle class 42d>  
}
```

<shapes/Square.java 41b>≡ 41b

```
public class Square extends Rectangle {  
    <constructors of the shapes/Square class 42c>  
    <methods of the shapes/Square class 43a>  
}
```

The minimal state needed to define a rectangle is its width and its height. A reasonable constructor takes both values and fills them accordingly. We'll maintain an invariant that side lengths are never negative, so if a negative length is given it will be replaced by zero.

42a *<fields of the shapes/Rectangle class 42a>*≡ (41a)

```
private int width;
private int height;
```

42b *<constructors of the shapes/Rectangle class 42b>*≡ (41a)

```
public Rectangle(int w, int h) {
    if (w < 0) { width = 0; } else { width = w; }
    if (h < 0) { height = 0; } else { height = h; }
}
```

When constructing a subclass, the first line of its constructor should call a constructor of its parent class, using the keyword “super”. As our square here has no additional fields of its own, this is all that is necessary.

42c *<constructors of the shapes/Square class 42c>*≡ (41b)

```
public Square(int length) { super(length, length); }
```

The interface we define for rectangles will have three queries: one for each field to read its value (called a *getter*) and one to get the area of the shape. It will also have three modification methods: one to set each field on its own, and one to set both at once. Each modification method must maintain the invariant that lengths be nonnegative. This could mean throwing an exception of some sort when given a negative value, or clamping the size to zero, or taking the absolute value. Any of these is reasonable if documented well. We'll go with the second, as it makes the method total and avoids having to deal with the edge-case that is the smallest representable int. We'll also override the `toString` method.

42d *<methods of the shapes/Rectangle class 42d>*≡ (41a)

```
public int getWidth() { return width; }
public int getHeight() { return height; }
public long getArea() { return (long)width * (long)height; }

public void setWidth(int w) {
    if (w < 0) { width = 0; } else { width = w; }
}
public void setHeight(int h) {
    if (h < 0) { height = 0; } else { height = h; }
}
public void setSize(int w, int h) { setWidth(w); setHeight(h); }

@Override
public String toString() {
    return "Rectangle(" + getWidth() + ", " + getHeight() + ")";
}
```

For a square, the queries are unchanged. However, the modification methods of the rectangle class do not enforce the square's additional invariant that the width and height are equal. The subclass can override methods of an ancestor class, and we can use this to guarantee that `setWidth` and `setHeight` maintain this invariant. For `setSize`, there is again a choice to be made. Should supplying mismatched lengths be treated as an error? Should one be prioritized and the other ignored? Something else? Here we'll use the convention that the higher value be used for both the height and the width. To call methods of the parent class, we again use the keyword `super`.

```

<methods of the shapes/Square class 43a>≡ (41b) 43a
    @Override
    public void setWidth(int w) {
        super.setWidth(w);
        super.setHeight(w);
    }
    @Override
    public void setHeight(int h) { setWidth(h); }
    @Override
    public void setSize(int w, int h) {
        if (w > h) { setWidth(w); } else { setWidth(h); }
    }

    @Override
    public String toString() {
        return "Square(" + getWidth() + ")";
    }

```

Consider the following program class:

```

<shapes/ShapeTest.java 43b>≡ (43b)
public class ShapeTest {
    public static void main(String[] args) {
        actOn(new Rectangle(5, 5));
        System.out.println();
        actOn(new Square(5));
    }
    public static void widen(Rectangle r) {
        int w = r.getWidth();
        if (w < Integer.MAX_VALUE) { r.setWidth(w + 1); }
    }
    public static void actOn(Rectangle r) {
        System.out.println(r);
        System.out.println("        Area: " + r.getArea());
        widen(r);
        System.out.println("Widened area: " + r.getArea());
    }
}

```

Because the modification methods belong to the square or rectangle, rather than to the class, a method that takes an arbitrary rectangle and increases its width by one unit can freely operate on a square without violating its invariants; the height will change as well for squares. The above program will report that the widened area for the rectangle is 30 (5×6), while that of the square is 36 (6×6). The correct method to use is determined at runtime, based on what kind of rectangle is actually received, rather than at compile time. Object-oriented programming allows a programmer to define a hierarchy of classes, where instances of a subclass can be operated on as if they were instances of an ancestor class, with confidence that class invariants are never broken.

4.2 Containers and Bounds

Consider an object that holds a reference to some other object. We can define a `Container` class that describes these wrapper objects. It might be reasonable for the wrapper to have a field of type `Object`, as this can reference any object of reference type, but this is difficult to work with: to wrap an object is easy enough, but in doing so you lose the ability to interact with it by any methods other than those of `Object`. Instead, using Java's *generics*, we can have a *type variable* that can be used in place of a type. To introduce a variable `T`, it is enclosed in angle-brackets like `<T>`. If multiple variables are needed, they are separated by commas within the angle-brackets.

44 `<Container.java 44>`≡

```
public class Container<T> {
    private T contents;
    private int replacements;
    public Container(T x) { contents = x; replacements = 0; }
    public T getContents() { return contents; }
    public void setContents(T x) { contents = x; replacements++; }
    public int getReplacements() { return replacements; }
}
```

This allows for *parametric polymorphism*: a single definition works, generically, for any type. This contrasts with *ad hoc polymorphism*, where the behavior for each applicable type is coded separately. In Java in particular, parametric polymorphism can only handle reference types; primitive types must be handled either through boxing and unboxing or through ad hoc polymorphism.

Type variables may also be introduced directly before the return type of a particular method. The type variables on a class are filled in when an instance is created, so for a static method (for which no instance is defined), any necessary variables must be introduced on the method itself. When using a class of generic type, such as when declaring a variable of that type, the angle brackets are written

with an actual type; they must also be supplied on the constructor, but if the actual type can be inferred at that point then it need not be restated.

<declaring and initializing a container 45a>≡ 45a

```
Container<Rectangle> c = new Container<>(new Rectangle(3, 4));
```

As we saw in the previous section, a method that wants a `Rectangle` can be given a `Square`. However, the same does not hold upon wrapping: a method that wants a `Container<Rectangle>` cannot be given a `Container<Square>` or vice versa. For instance, a method that stuffs a particular rectangle into a container would be trying to use a `Rectangle` as a `Square` if given a `Container<Square>`!

<sample container methods 45b>≡ 45b

```
public static void insert2x4(Container<Rectangle> r) {
    r.setContents(new Rectangle(2, 4));
}
```

If putting something into the container, it must be a subtype of the wrapped type. If taking something out of the container, it can be used where a supertype is expected. If a container is used for both input and output, then only the exact type is valid.

We can use *wildcards* to describe the desired usage. To put a rectangle into a container, it must be a container of some ancestor class of `Rectangle`: `Container<? super Rectangle>`. The question mark is a wildcard that on its own represents any type, and the “`super Rectangle`” is a *bound*, restricting to ancestor classes of `Rectangle`. A container from which a rectangle is extracted would instead be `Container<? extends Rectangle>`. Here, “`extends Rectangle`” bounds the wildcard in the other direction, restricting to subclasses of `Rectangle`. If the container is used for both input and output, it should just be `Container<Rectangle>`, as it is tightly bounded. Finally, an unbounded wildcard is used when the contained object is neither used nor replaced. The container defined here tracks how many times its contents have been replaced, so a method could use `Container<?>` if it cared only about this update count. If the need arises, it is also possible to introduce type variables with bounds like `<T extends C>` or `<T super C>` to limit `T` to sub- or superclasses of `C`, respectively.

4.3 Type Variance

As we have noted, a read–write container must have exactly the parameter type expected; `Container<T>` is *invariant* on `T`. A type is *covariant* on its parameter if it preserves the subtype relation, *contravariant* on its parameter if it reverses this relation, and *invariant* if only the exact parameter is allowed.

A container from which values are extracted as a supertype can actually be a container of subtype values: `Container<? extends T>` is *covariant* on `T`. While you cannot assign a `Container<Square>` to a variable with type restriction `Container<Rectangle>`, you can assign a `Container<? extends Square>` to

a variable with type restriction `Container<? extends Rectangle>`; the former is a subclass of the latter. You can read “`? extends`” as “source of”. A source of squares is a source of rectangles, so this is valid.

If items are to be inserted into the container but not read back out, the items can be a subtype of the container’s declared type: `Container<? super T>` is *contravariant* on `T`. You can assign a `Container<? super Rectangle>` to a variable with type restriction `Container<? super Square>`; the former is a subclass of the latter. You can read “`? super`” as “sink for”. A container that accepts rectangles (a sink for rectangles) is a container that accepts squares (a sink for squares).

Java supports a notion of covariant return types: if a method of a superclass returns an object of type `T`, then the overriding method of a subclass can return any subtype of `T`. This allows the subclass to make stronger guarantees about its output. It would be reasonable to allow contravariant parameter types as well; if a superclass takes a value of type `T`, then the overriding method of a subclass should be able to take any supertype of `T` and still be valid. However, Java does not directly allow for this: such a method can be written, but it will not override the parent class method. So a method with the same parameter types as in the parent class should still be written, but it can simply be a redirect to the broader method.

4.4 Behavioral Subtyping

The idea of class hierarchies is that an instance of a subclass can be supplied as if it were an instance of a superclass and the program will still work correctly. This is why the bounding criteria are what they are for our containers: if an object is being inserted, it can be a subtype of the contained value’s type, while if an object is being extracted and inspected, it can be treated as if it were any supertype. These behaviors are distinct and require distinct handling. The concept of *behavioral subtyping* formalizes this notion of *substitutability*, where a good subclass preserves every observable property of its parent superclass.

This implies stronger requirements on subtyping than merely having one type be a mathematical subset of the other. Preconditions on operations should be no stronger in the subclass; all valid inputs to the superclass method should be usable on the subclass. Postconditions on operations should be no weaker in the subclass; the outputs from the subclass must be compatible with how outputs from the superclass could be used. As invariants are properties that are simultaneously pre- and postconditions, a subclass should not enforce stronger invariants than its parent superclass, nor should it have weaker invariants. This suggests that having a mutable `Square` as a subclass of mutable `Rectangle` is not actually a good design.

In a `Rectangle`, it is observable that `setSize(3,4)` results in having `getWidth()` return 3, in having `getHeight()` return 4, and in having `getArea()`

return 12. There is no `Square` for which `getArea()` returns 12, nor is there a `Square` where `getWidth()` would be 3 and simultaneously `getHeight()` would be 4. If, however, the `Square` and `Rectangle` were persistent data structures that returned new rectangles instead of modifying their own internals, then there would be no issue. So whether one class is a good subclass of another depends on what operations should be available!

A persistent rectangle and square class are provided here, implementing the same queries as before. The same modifications are provided, except, because the classes are written in persistent style, these methods return a new `Rectangle` instead of modifying the source. Most modifications of a square still return an arbitrary rectangle in order to maintain the expected properties. Finally, a new modification method, `rotate`, is provided to demonstrate that the subclass can still provide stronger guarantees.

<shapes/persistent/Rectangle.java 47>≡

47

```
public class Rectangle {
    private final int width;
    private final int height;

    public Rectangle(int w, int h) {
        if (w < 0) { width = 0; } else { width = w; }
        if (h < 0) { height = 0; } else { height = h; }
    }

    public int getWidth() { return width; }
    public int getHeight() { return height; }
    public long getArea() { return (long)width * (long)height; }

    public Rectangle setWidth(int w) {
        return new Rectangle(w, getHeight());
    }
    public Rectangle setHeight(int h) {
        return new Rectangle(getWidth(), h);
    }
    public Rectangle setSize(int w, int h) {
        return setWidth(w).setHeight(h);
    }
    public Rectangle rotate() {
        return new Rectangle(getHeight(), getWidth());
    }

    @Override
    public String toString() {
        return "Rectangle(" + getWidth() + "," + getHeight() + ")";
    }
}
```

```

48 <shapes/persistent/Square.java 48>≡
    public class Square extends Rectangle {
        public Square(int length) { super(length, length); }

        @Override
        public Square rotate() { return this; }
    }

```

Although `setWidth`, `setHeight`, and `setSize` must return an unconstrained rectangle to avoid changing provable behavior, the `rotate` method can provide stronger guarantees on a square: rotating a square results in exactly the same square. This demonstrates that overrides are not incompatible with good subtyping. The `toString` method is a bit of an odd consideration: if the square were to output using a different format, as in the earlier examples, then that would change provable behavior, so it is not further overridden here.

4.5 Summary

A subclass is defined using the `extends` keyword, which can be read as “is a”. A variable restricted to some type *C* can have any instance of any subclass of *C* assigned into it. This includes, for example, parameters to method calls. Similarly, if the parent superclass returns a value of type *C*, then the subclass can return any subclass of *C*.

A class can be parameterized by a type using angle-brackets; if instead of “`public class LinkedList`” we had defined our linked lists in the previous chapter as “`public class LinkedList<E>`” (and similarly for its internal `Node` class), and if we had used *E* everywhere that we had used `Integer`, then the list would have parametric polymorphism. With it, you could make a list of integers as a `LinkedList<Integer>`, a list of rectangles as a `LinkedList<Rectangle>`, and so on. Each of these is a list of one kind of thing, but you decide when declaring the list what kind of thing that is.

When a type is a parameter like this, it cannot, by default, be replaced by a subclass. This is because we do not necessarily know whether the list is to be used as a source, as a sink, both, or neither. After making these modifications to the list class, the following statements hold. A `LinkedList<? extends Rectangle>` is a source of rectangles, which might be squares. A `LinkedList<? super Square>` is a sink for squares, which might also accept rectangles. A `LinkedList<?>` cannot accept or produce any values, but can still tell you its size, for example.

When designing class systems in an object-oriented system, you should consider not only the mathematical subset relationship on the represented values, but also the observable behaviors of the system. Instances of a subclass should be substitutable for instances of the parent superclass without changing observable behavior.

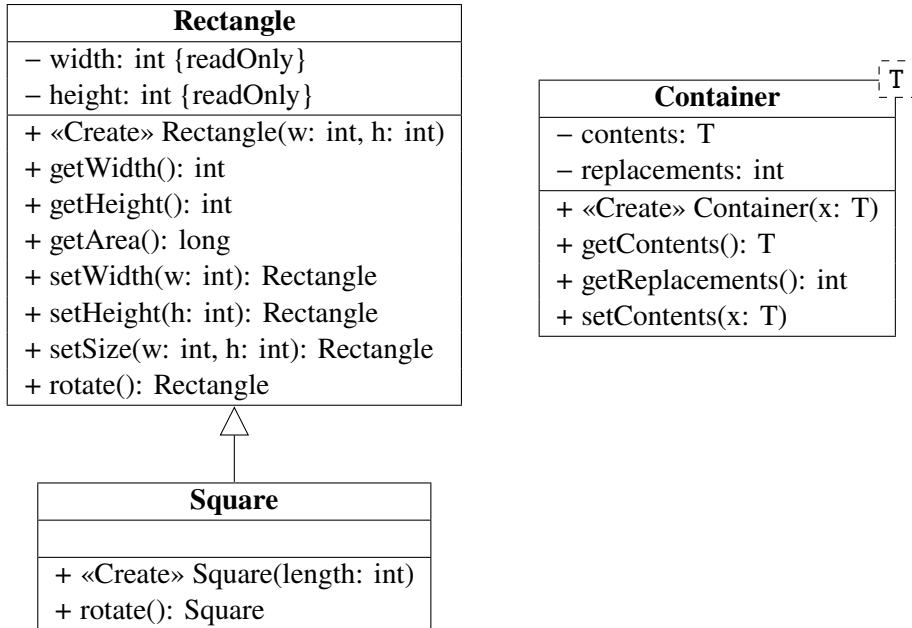


Figure 4.1: A UML class diagram for some classes described in this chapter.

Persistent data structures, being immutable, require less thought in this regard. Ephemeral structures have more kinds of observable behaviors and therefore require more care to design good subclass relations. In a later chapter, we will discuss interfaces, which describe behavior in the abstract without concrete implementation. Judicious use of interfaces and subtyping will help to design good object-oriented systems.

Figure 4.1 depicts a UML class diagram for the persistent **Rectangle** and **Square** classes detailed in the previous section, as well as the generic **Container** class. The arrow with the open triangular arrowhead means that **Square** is a subclass of **Rectangle**. The class is divided into three sections: the name of the class (centered at the top), the fields of the class, and the methods of the class. There are no static fields or static methods here, but if there were, their names would be underlined. The dotted box in the upper right of the container indicates that this class is parameterized by the type variable `T`. Fields and methods have their visibility indicated by a sign on the left: `-` for **private**, `+` for **public**, and `#` for **protected**. Type restrictions are specified after the things they apply to, separated by a colon; a method with no type specified in the diagram returns nothing. Finally, `«Create»` annotates a constructor, and `{readOnly}` annotates **final** variables. This is far from a complete listing of what can be depicted in a UML class diagram.